Python Data Science Stack Guide

NumPy, pandas, and Matplotlib Crash Course

Implementing machine learning algorithms requires a solid coding stack. In Python, the standard stack consists of NumPy (for matrix algebra and array manipulation), pandas (for tabular data loading, manipulation, and cleaning), and Matplotlib (for data visualization and plotting metrics). This guide covers core functions, broadcasting rules, data structures, and figure customization layouts.

1. NumPy: High-Performance Vectorized Array Algebra

NumPy provides the fundamental N-dimensional array object, ndarray. Unlike standard Python lists, NumPy arrays store elements contiguously in memory, allowing for vectorized computations compiled in C. Vectorization avoids standard Python for loops, accelerating math computations by several orders of magnitude.

A key concept in NumPy is **Broadcasting**. When performing operations on arrays of different shapes, NumPy automatically stretches the dimensions of the smaller array to match the larger array (see Figure 1). A dimension is stretchable if it is exactly 1 or matches the other array's dimension.

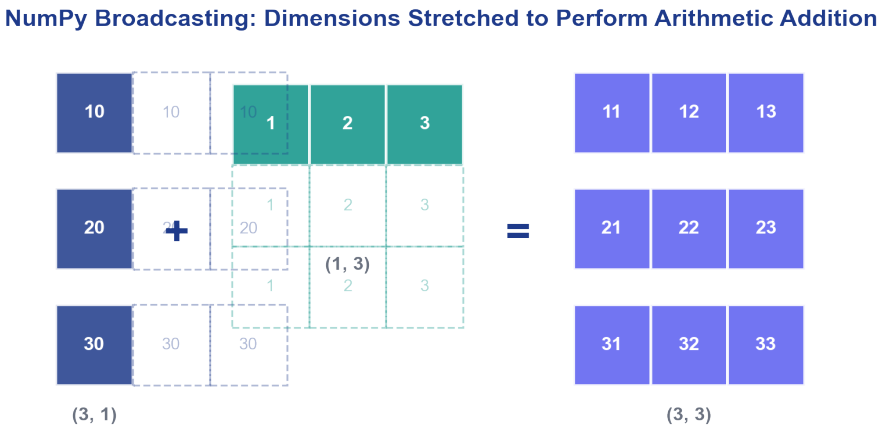

Figure 1: Stretching dimensions during NumPy element-wise array operations.

2. pandas: Structured Tabular Data Preprocessing

pandas introduces the DataFrame (a 2D table with labeled axes) and the Series (a 1D labeled array). pandas is the primary tool for data preprocessing: loading files (e.g. CSVs, SQL queries), indexing data using .loc (label-based) or .iloc (index-based), handling missing data (NaNs), performing groupings (groupby) for aggregations, and merging data tables via database-style joins.

In machine learning, raw datasets are loaded into DataFrames, missing inputs are imputed, categoricals are encoded, and the final values are converted into NumPy arrays (using df.values or .to_numpy()) before being passed to algorithms.

3. Matplotlib: Beautiful and Customized Data Visualizations

Matplotlib is the core plotting library in Python. It uses an object-oriented API where a Figure represents the canvas, and Axes represent the actual plots. This allows you to construct and customize multi-panel visualizations (subplots), add titles, configure axes scales, customize gridlines, and format legends for publication-grade quality (see Figure 2).

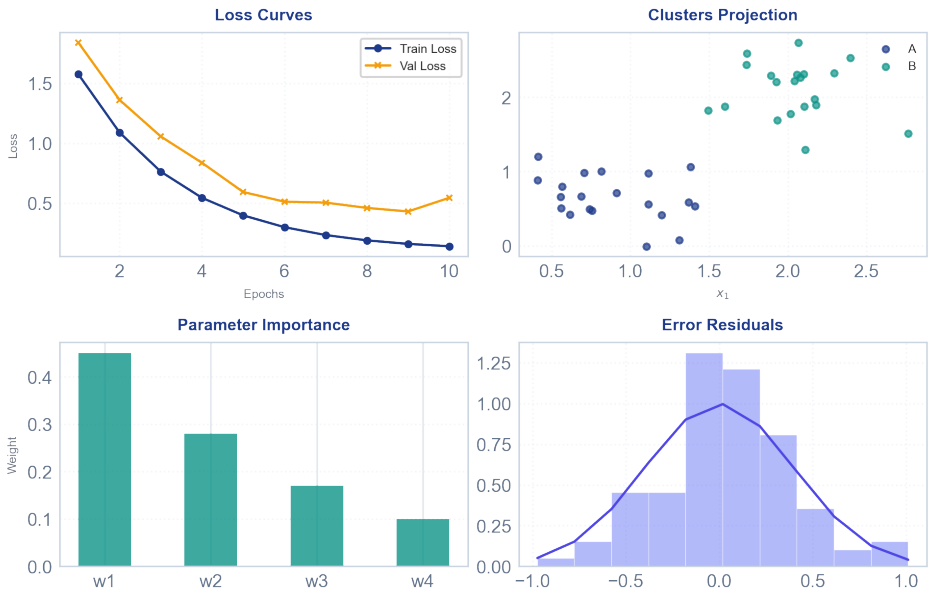

Figure 2: Object-oriented subplots illustrating training losses, datasets, and distributions.

4. Machine Learning Integration: End-to-End Pipeline

A typical ML data pipeline loads raw data with pandas, prepares features, normalizes variables using scaling equations ($x_{scaled} = \frac{x - \mu}{\sigma}$), fits a linear regression model via the normal equation ($\theta = (X^T X)^{-1} X^T y$), and plots the results. Here is the full code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# 1. Load data with pandas
df = pd.read_csv('dataset.csv')
df = df.dropna(subset=['target', 'feature'])

# 2. Extract features and convert to NumPy
X_raw = df['feature'].to_numpy().reshape(-1, 1)
y = df['target'].to_numpy()

# 3. Scale features (Calculus / Stats normalization)
X_scaled = (X_raw - X_raw.mean()) / X_raw.std()
X = np.hstack((np.ones((len(X_scaled), 1)), X_scaled)) # Add intercept bias column

# 4. Fit regression model using Normal Equation (Linear Algebra)
theta = np.linalg.inv(X.T @ X) @ X.T @ y

# 5. Plot model predictions with Matplotlib
plt.scatter(X_scaled, y, color='#1E3A8A', alpha=0.5, label='Actual Data')
plt.plot(X_scaled, X @ theta, color='#0D9488', lw=2, label='Model Fit')
plt.title('Feature vs. Target regression')
plt.legend()
plt.savefig('result.png')
```

ML INSIGHT: *Vectorized calculations in NumPy are faster because they avoid Python interpreter overhead and utilize SIMD (Single Instruction, Multiple Data) CPU registers. When writing code, always seek to vectorize your equations and avoid nested loop iterations.*

Python Data Science Stack Quick Reference Sheet

Core commands, functions, and coding templates for data manipulation.

Library	Common Operations & Syntax	Description / Context
NumPy Creation	np.array(list) np.zeros(shape) np.ones(shape) np.arange(start, stop, step) np.linspace(start, stop, num)	Initializes matrices, constant arrays, and linear spacing arrays.
NumPy Matrix Algebra	a.T # Transpose np.dot(a, b) or a @ b # Dot product np.linalg.inv(a) # Inverse np.linalg.svd(a) # SVD	Algebraic operations for calculating weights and principal components.
NumPy Math Methods	a.mean(axis) a.std(axis) a.sum(axis) np.exp(a) np.log(a)	Aggregate metrics along columns (axis=0) or rows (axis=1).
pandas Selection	df['col'] # Select column df.loc[rows, cols] # Label indexing df.iloc[i, j] # Numeric indexing df[df['c'] > 5] # Masking	Accesses rows, columns, and conditional subsets of DataFrames.
pandas Preprocessing	df.isnull().sum() df.fillna(value) df.dropna() df.drop_duplicates()	Data cleaning, handling missing inputs, and dropping duplicates.
pandas Merge / Join	pd.merge(df1, df2, on='key', how='inner') pd.concat([df1, df2], axis=0)	Joins datasets based on shared keys or concatenates arrays.
pandas Aggregation	df.groupby('col').mean() df.describe() df.value_counts()	Computes group summaries and counts of categorical inputs.
Matplotlib Figure Setup	fig, ax = plt.subplots(r, c, figsize=(w, h)) ax.set_title() ax.set_xlabel() ax.set_ylabel()	Initializes object-oriented canvas grid and axis labels.
Matplotlib Plot Types	ax.plot(x, y, label) ax.scatter(x, y, color) ax.hist(data, bins) ax.bar(categories, heights) ax.legend()	Draws custom lines, scatter clusters, histograms, and bar charts.